Title: Phase 2 Backend Implementation & API Design

1. Architectural Overview

- **Tech Stack:** Node.js, Express.js, and raw MySQL.
- **Structure:** We used a standard MVC-style architecture, separating our code into `routes`, `controllers`, `middleware`, and `config` to ensure clean, maintainable code for Phase 3.
- **Database Connection:** We implemented a MySQL Connection Pool (`mysql2/promise`) in `config/db.js` to efficiently handle multiple database connections without crashing.

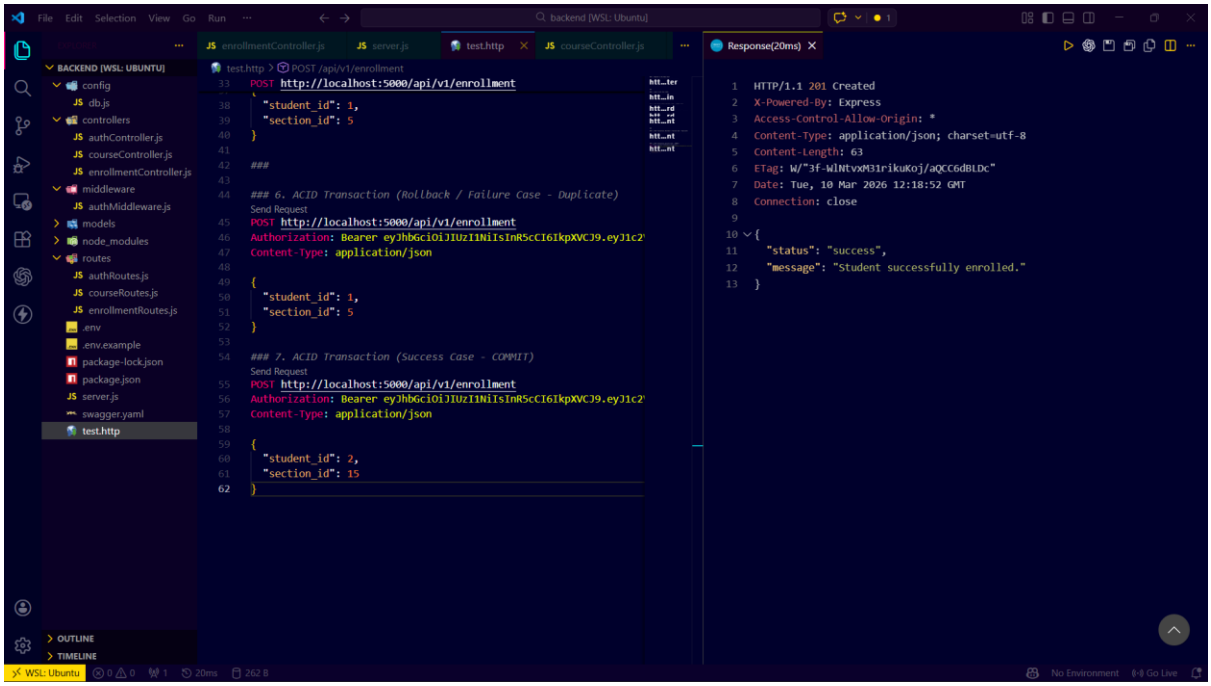
2. Security & RBAC (Role-Based Access Control)

- **Authentication:** Passwords are encrypted using `bcrypt` before hitting the database. We implemented JSON Web Tokens (JWT) for stateless authentication.
- **Authorization (RBAC):** We created custom middleware (`authMiddleware.js`) that intercepts requests, reads the JWT, and verifies if the user's role (`Admin`, `Instructor`, or `Student`) is authorized to access specific endpoints (e.g., only Admins/Students can trigger enrollments).

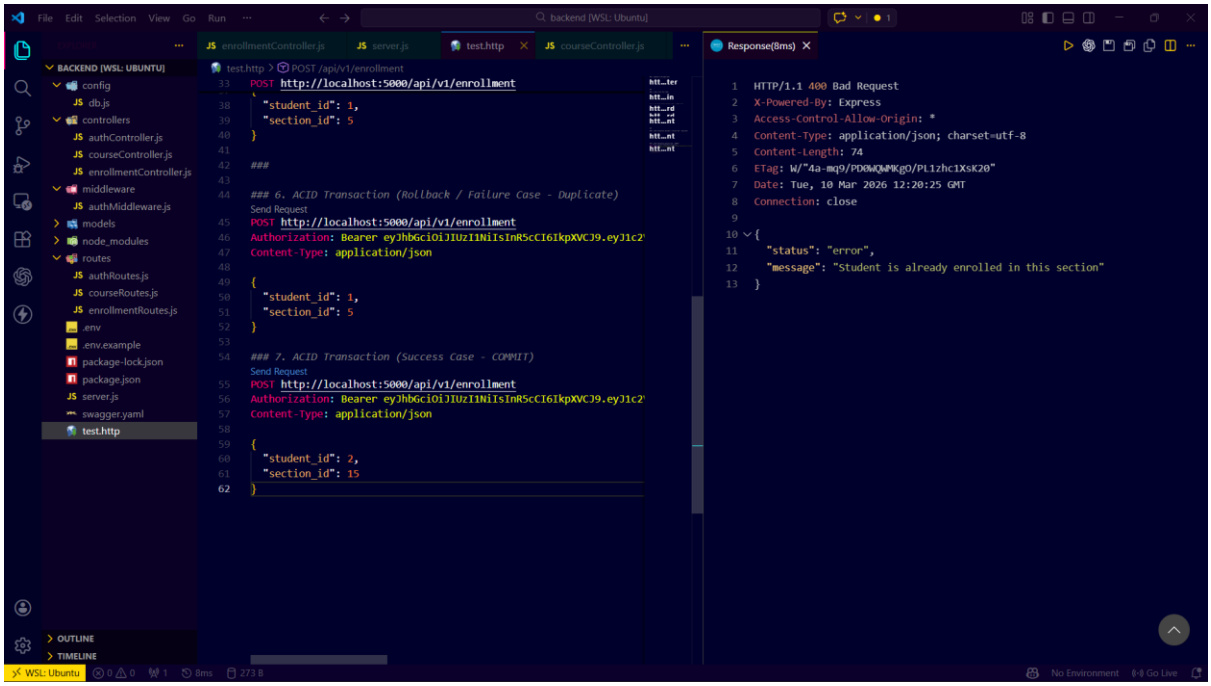
3. Transaction Management (ACID Compliance)

- **Implementation:** We successfully implemented a strict ACID transaction for the Course Enrollment process using raw SQL (`BEGIN`, `COMMIT`, `ROLLBACK`).
- **Scenario:** When a student attempts to enroll, the system locks the section row (`FOR UPDATE`) to check capacity, checks for duplicate enrollments, and inserts the record. If any check fails, the transaction is immediately rolled back to prevent dirty reads or data corruption.
- **Proof:**

  Once enrolled:

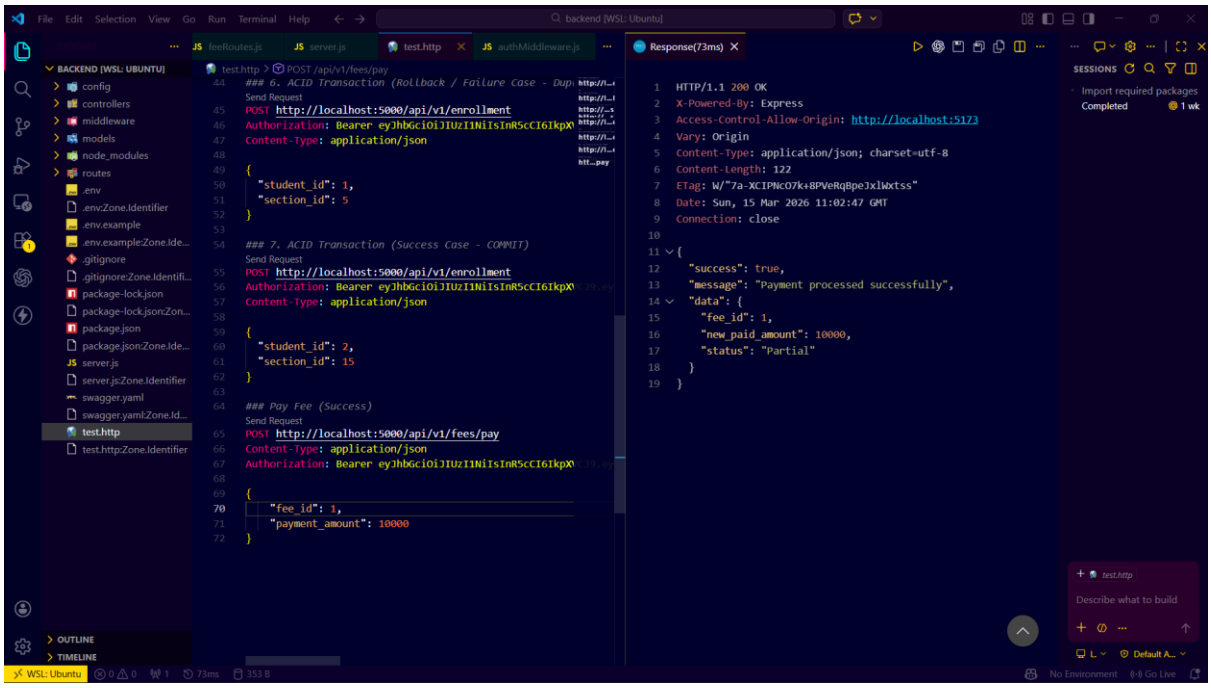


Cannot enroll again:

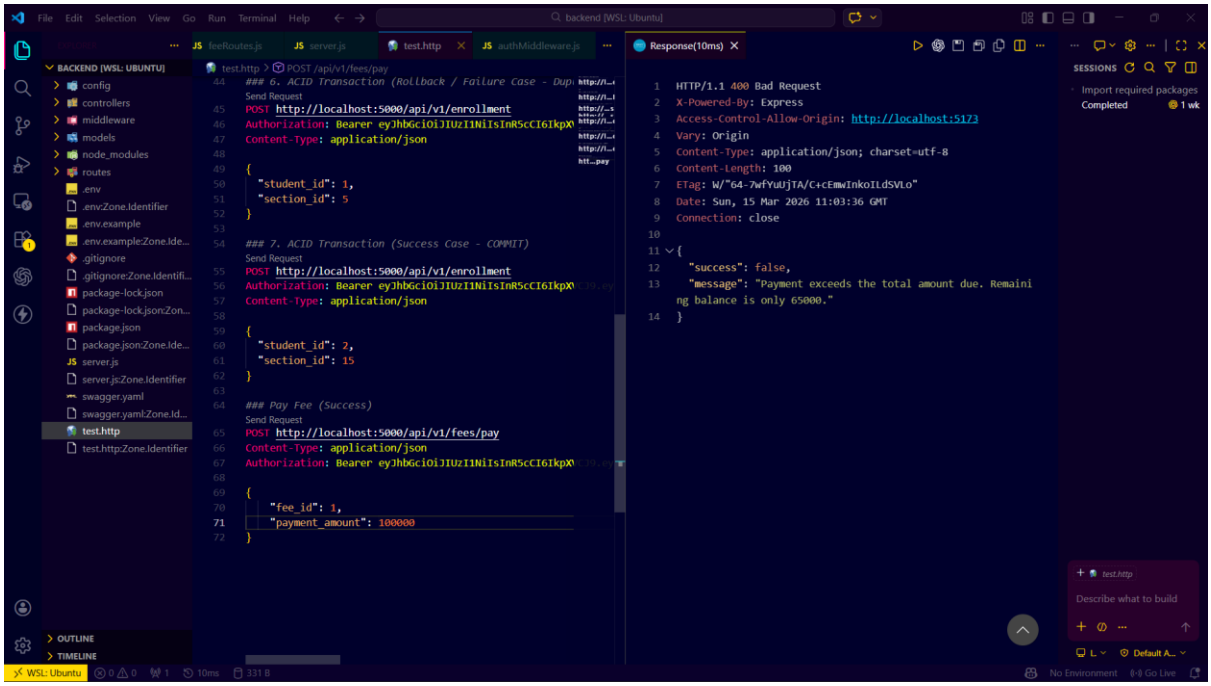


```
--- Transaction Started for Student 1, Section 5 ---
Capacity check passed.
--- TRANSACTION FAILED & ROLLED BACK --- Reason: Student is already enrolled
 in this section


--- Transaction Started for Student 2, Section 15 ---
Capacity check passed.
Duplicate check passed.
Enrollment record inserted.
--- Transaction Committed Successfully! ---


--- Transaction Started for Student 2, Section 15 ---
Capacity check passed.
--- TRANSACTION FAILED & ROLLED BACK --- Reason: Student is already enrolled
 in this section
```

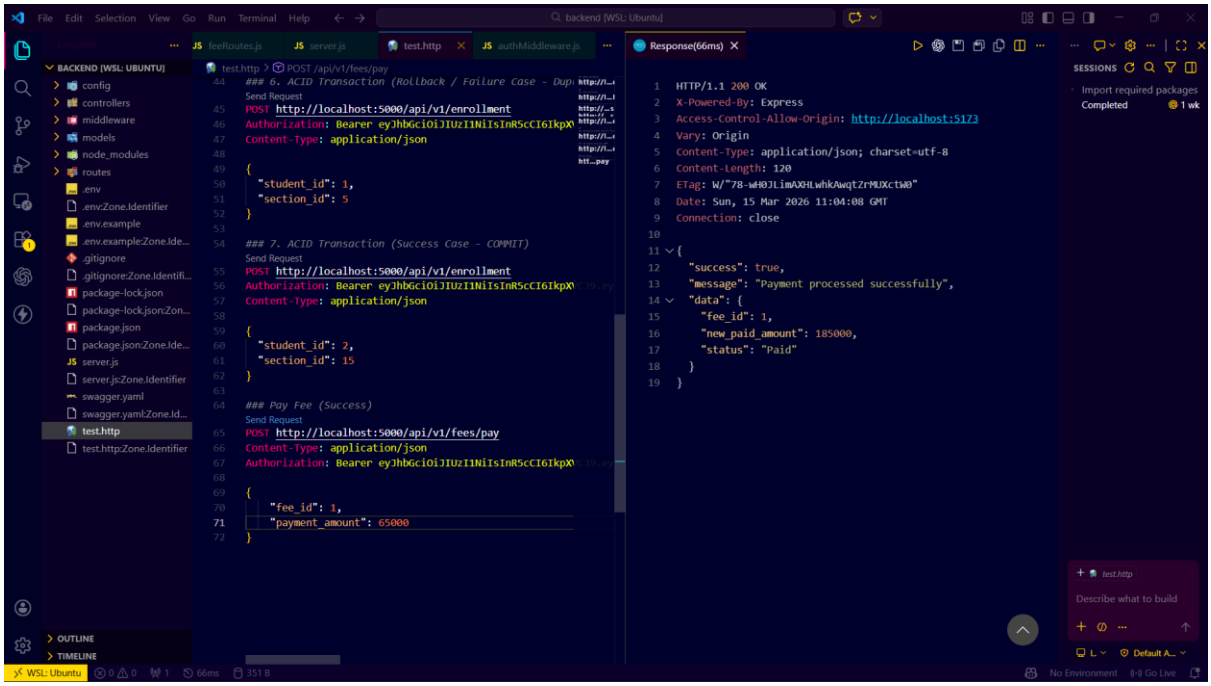
Partial payment made:



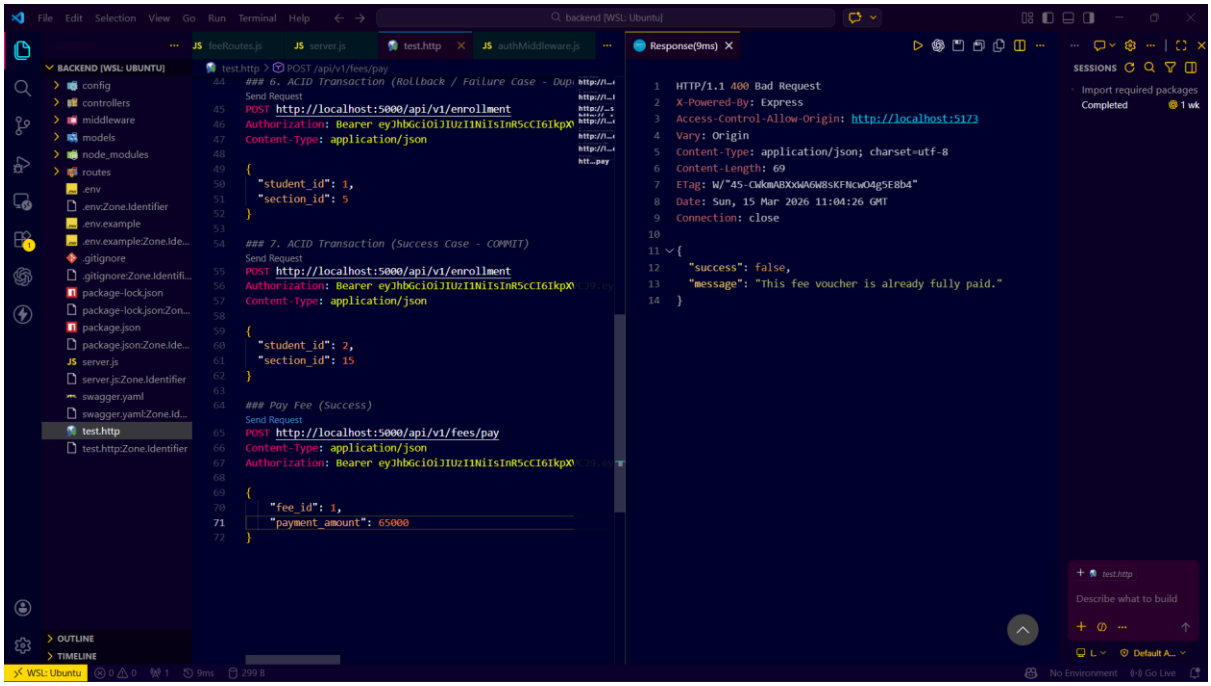
Can't exceed the due amount:



Full payment done:



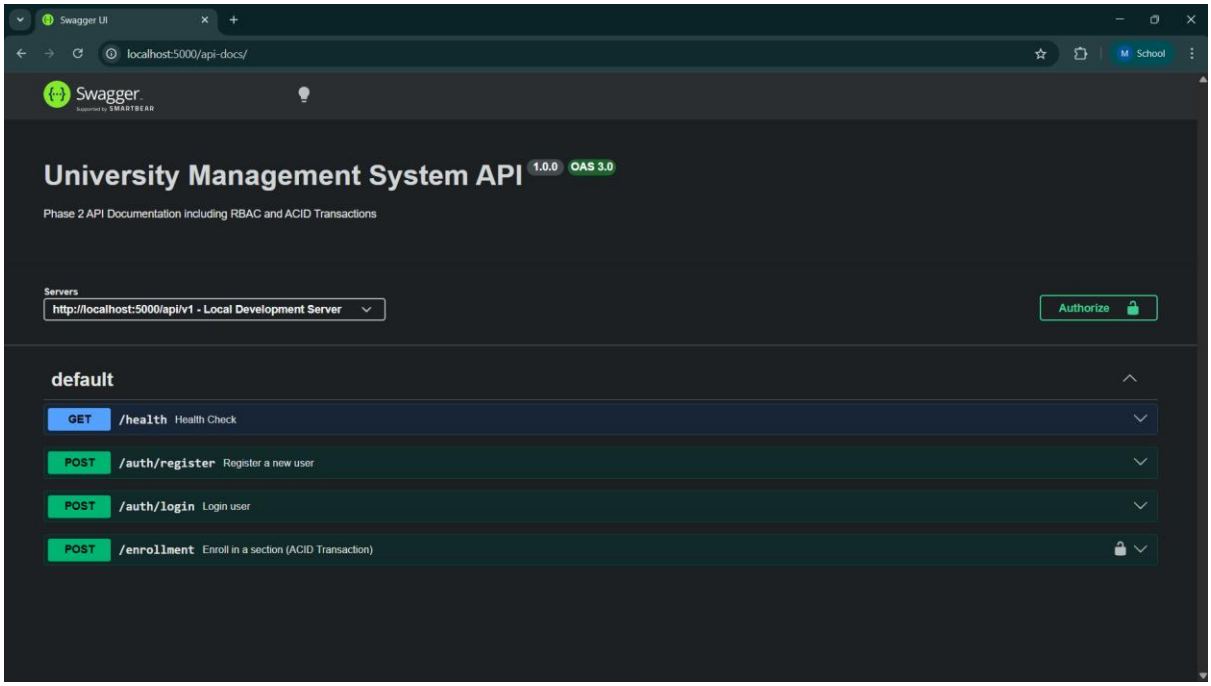
Once voucher is paid can’t pay more:



```
Ubuntu

--- STARTING FEE PAYMENT TRANSACTION (Fee ID: 1) ---
1. Transaction Began
--- TRANSACTION FAILED & ROLLED BACK ---
Reason: This fee voucher is already fully paid.


--- STARTING FEE PAYMENT TRANSACTION (Fee ID: 1) ---
1. Transaction Began
2. Updating fee record. New Status: Partial
--- TRANSACTION COMMITTED SUCCESSFULLY ---


--- STARTING FEE PAYMENT TRANSACTION (Fee ID: 1) ---
1. Transaction Began
2. Updating fee record. New Status: Partial
--- TRANSACTION COMMITTED SUCCESSFULLY ---


--- STARTING FEE PAYMENT TRANSACTION (Fee ID: 1) ---
1. Transaction Began
2. Updating fee record. New Status: Partial
--- TRANSACTION COMMITTED SUCCESSFULLY ---


--- STARTING FEE PAYMENT TRANSACTION (Fee ID: 1) ---
1. Transaction Began
--- TRANSACTION FAILED & ROLLED BACK ---
Reason: Payment exceeds the total amount due. Remaining balance is only 65000.


--- STARTING FEE PAYMENT TRANSACTION (Fee ID: 1) ---
1. Transaction Began
2. Updating fee record. New Status: Paid
--- TRANSACTION COMMITTED SUCCESSFULLY ---


--- STARTING FEE PAYMENT TRANSACTION (Fee ID: 1) ---
1. Transaction Began
--- TRANSACTION FAILED & ROLLED BACK ---
Reason: This fee voucher is already fully paid.
```

4. API Specification

- **Documentation:** We utilized `swagger-ui-express` and `yamljs` to generate an interactive OpenAPI 3.0 documentation page.
- **Proof:**


Swagger UI
localhost:5000/api-docs/
School
Swagger
University Management System API
1.0.0
OAS 3.0
Phase 2 API Documentation including RBAC and ACID Transactions
Servers
http://localhost:5000/api/v1 - Local Development Server
Authorize
default
GET /health Health Check
POST /auth/register Register a new user
POST /auth/login Login user
POST /enrollment Enroll in a section (ACID Transaction)